

Feature Engineering for Beginners: Making Your Data Model-Ready

 medium.com/codetodeploy/feature-engineering-for-beginners-making-your-data-model-ready-6696d8e267c2

Gitanjali

January 5, 2026

The First Publication That Blends Tech Insights + Real Job Opportunities



data.
engineer

Before a machine learning model learns anything useful, it looks at your data and asks a simple question: *What am I supposed to work with?*

Feature engineering is the step where you answer that question.

It sits quietly between data cleaning and modeling, often underestimated, sometimes rushed, and yet responsible for a huge portion of a model's final performance. You can have a powerful algorithm, clean data, and still get poor results if your features don't represent the underlying problem well.

In this article, we'll walk through feature engineering step by step using Python. The goal is not to memorize techniques, but to understand *how to think* about transforming raw data into something a model can actually learn from.

We'll use the Titanic dataset again, not because it's exciting, but because it's familiar. That way, we can focus on ideas instead of context.

What Is Feature Engineering?

A *feature* is simply an input variable used by a model.

Feature engineering is the process of:

- Creating new features from existing data
- Transforming features into more useful forms
- Removing features that confuse or mislead the model

It is not about adding more columns blindly. It is about representing reality in a way machines can understand.

Load the Dataset

```
seaborn sns pandas pd numpy npdf = sns.()
```

Before engineering anything, you should already have done basic cleaning: handling missing values, fixing text labels, and removing obvious errors.

Feature engineering builds on that foundation.

1. Understanding Feature Types

Before touching the data, pause and classify columns mentally.

- Numerical: age, fare
- Categorical: sex, embarked, class
- Binary: survived, adult_male
- Date or time-based (not here, but common)
- Derived or redundant features

This classification guides what transformations make sense.

2. Encoding Categorical Variables

Machine learning models do not understand text. They only understand numbers.

Binary Encoding

For columns like `sex`, binary encoding works well.

```
[] = [].map({: 1, : 0})
```

Simple, readable, effective.

One-Hot Encoding

For features with more than two categories:

```
= pd.get_dummies(df, columns=[], drop_first=)
```

This creates separate columns like `embarked_Q` and `embarked_S`.

Be careful: one-hot encoding can explode feature count if a column has many unique values.

3. Creating Meaningful Features

Good features often come from domain thinking, not libraries.

Family Size

Instead of using `sibsp` and `parch` separately:

```
[] = [] + [] + 1
```

This captures social grouping better than raw counts.

Traveling Alone

```
[] = ([] == 1).astype(int)
```

Sometimes a simple boolean feature is more powerful than multiple numeric ones.

4. Binning Continuous Variables

Continuous values can hide patterns.

Age Groups

```
[ ] = pd.cut( [ ], bins=[0, 12, 18, 35, 60, 100], labels=[ , , , , ])
```

This turns raw age into life stages, which can be more meaningful for some models.

You can later encode this feature for modeling.

5. Transforming Skewed Features

Many real-world numerical features are skewed.

Let's look at **fare**.

```
[ ] = np.log1p([ ])
```

Log transformation compresses extreme values and often helps linear models perform better.

This is not a trick. It is a mathematical way of telling the model: large values exist, but don't let them dominate everything.

6. Handling Missing Values Intelligently

Instead of just filling missing values, sometimes *missingness itself* is a signal.

```
[ ] = [ ].isna().astype(int)
```

Now the model can learn whether missing age carries information.

This is especially useful when data is not missing randomly.

7. Dropping Redundant or Leaky Features

Not all columns deserve to stay.

Examples:

- IDs
- Features created after the outcome
- Multiple columns describing the same thing

```
= df.drop(columns=[ ])
```

Feature engineering is as much about removal as creation.

8. Scaling Numerical Features

Many models perform better when numerical features are scaled.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df = scaler.fit_transform(df)
```

Scaling does not add information.

It makes learning easier.

9. Feature Engineering Is Iterative

There is no “final” feature set.

You create features.

You train a model.

You evaluate.

You refine.

Bad performance often points back to feature representation, not the algorithm.

Wrap Up

Feature engineering is the bridge between raw data and meaningful learning.

The key steps are:

- Understand feature types
- Encode categories thoughtfully
- Create features that reflect real-world logic
- Transform skewed values
- Use missingness as information
- Remove misleading columns
- Scale when the model requires it
- Iterate based on results

Good models are built on good features. Algorithms amplify what you give them.

References

Pandas Documentation

<https://pandas.pydata.org/docs/>

Scikit-Learn Feature Engineering Guide

<https://scikit-learn.org/stable/modules/preprocessing.html>

Kaggle Feature Engineering Course

<https://www.kaggle.com/learn/feature-engineering>

Applied Machine Learning by Andreas Müller

<https://amueller.github.io/>

Seaborn Titanic Dataset

<https://github.com/mwaskom/seaborn-data>

Thank you for being a part of the community

Before you go:



CodeToDeploy

👉 Be sure to **clap** and **follow** the writer 🙌

👉 Follow us: [Linkedin](#) | [Medium](#)

👉 CodeToDeploy Tech Community is live on Discord — [Join now!](#)

Note: This Post may contain affiliate links.